C++ COMPLETE REFERENCE LIBRARY • MODULE 08

08 DYNAMIC MEMORY ALLOCATION — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 stack vs heap and new delete
3. **Theory Chapter:** 02 dynamic arrays and objects
4. **Theory Chapter:** 03 memory bugs and raii ownership
5. **Visual Memory & Concept Diagrams** (2 Vector Diagrams)
6. **Annotated C++ Source Programs** (4 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

08 — Dynamic Memory Allocation

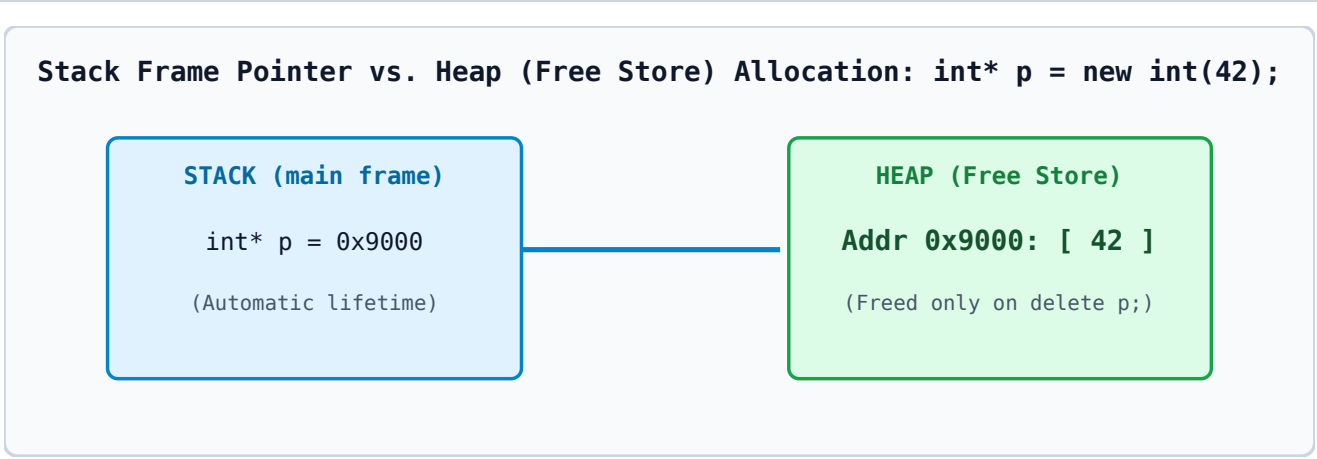
This module explains stack vs. free-store (heap) memory layout, scalar `new` / `delete`, array `new[]` / `delete[]`, memory leaks, dangling pointers, double deletion, and modern RAII alternatives (`std::unique_ptr` and `std::vector`).

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_stack_vs_heap_and_new_delete.md` (./01_THEORY/01_stack_vs_heap_and_new_delete.md)
  - `02_dynamic_arrays_and_objects.md` (./01_THEORY/02_dynamic_arrays_and_objects.md)
  - `03_memory_bugs_and_raii_ownership.md` (./01_THEORY/03_memory_bugs_and_raii_ownership.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_scalar_and_array_new_delete.cpp` (./02_PROGRAMS/01_scalar_and_array_new_delete.cpp) (Category A)
  - `02_dynamic_classroom_person_objects.cpp` (./02_PROGRAMS/02_dynamic_classroom_person_objects.cpp) (Category A)

  - `03_mismatched_delete_ub.cpp` (./02_PROGRAMS/03_mismatched_delete_ub.cpp) (Category D)
  - `04_modern_raii_vs_raw_new.cpp` (./02_PROGRAMS/04_modern_raii_vs_raw_new.cpp) (Category A)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**:
  - `diagram_01_stack_vs_heap_memory.svg` (./05_DIAGRAMS/diagram_01_stack_vs_heap_memory.svg)
  - `diagram_02_dangling_and_leak.svg` (./05_DIAGRAMS/diagram_02_dangling_and_leak.svg)

01 — Stack vs. Heap Memory and `new` / `delete`



Stack vs Heap Memory

1. Automatic Storage (Stack) vs. Dynamic Storage (Free Store / Heap)

Property	Stack Memory (Automatic)	Heap / Free Store (Dynamic)
Allocation Mechanism	Automatic when entering block scope `{}`	Explicit via `new` or `new[]`
Deallocation	Automatic LIFO pop when leaving block scope	Explicit via `delete` or `delete[]` (or RAII destructor)
Size Requirement	Must be known at compile time (`int arr[10];`)	Can be determined at runtime (`new int[n];`)
Lifetime	Tied strictly to the enclosing block scope	Persists until explicitly freed (`delete`)
Failure Mode	Stack overflow if recursion/arrays exceed stack limit	Throws `std::bad_alloc` if heap is exhausted

2. How `new` and `delete` Work Internally

1. **`T* p = new T(args);`**:
   - Step 1: Calls `operator new(sizeof(T))` to allocate raw bytes on the free store.
   - Step 2: Invokes the constructor `T::T(args)` on that memory block.
   - Step 3: Returns a typed pointer `T*`.
2. **`delete p;`**:
   - Step 1: Invokes the destructor `p→~T()` (if `T` is a class type).
   - Step 2: Calls `operator delete` to return the raw memory to the free store.
   - *(Note: In C++, `delete nullptr;` is guaranteed by the standard to be a safe no-op!)*

02 — Dynamic Arrays (`new[]` / `delete[]`) and Dynamic Objects

1. Allocating and Freeing Dynamic Arrays

CPP

```cpp
int n = 5;
int* arr = new int[n]{10, 20, 30, 40, 50};
// use arr[0] .. arr[4]
delete[] arr; // MUST use delete[], NEVER scalar delete!
arr = nullptr;
```

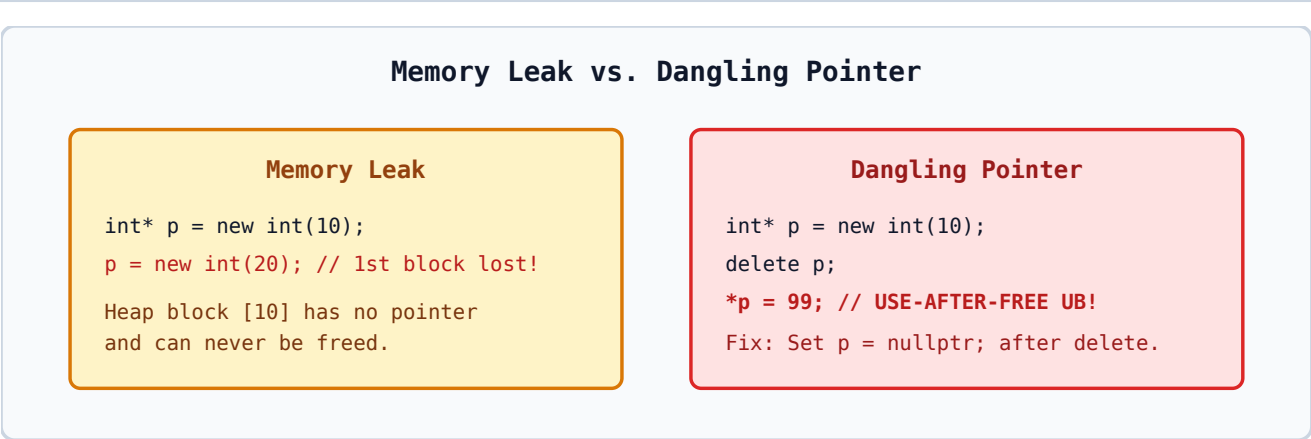
Strict Rule: Matching `new[]` with `delete[]` (`[expr.delete]`)

- `new T` **must** be paired with `delete ptr;`.
- `new T[n]` **must** be paired with `delete[] ptr;`.
- Calling scalar `delete ptr;` on memory allocated with `new T[n]` is **Undefined Behavior** (among other failures, destructors for elements `1..n-1` will not be called, and heap metadata may be corrupted).

2. Dynamic Classroom Objects

When `person* list = new person[3];` is executed, the **default constructor** of `person` is called 3 times (once for each element). When `delete[] list;` is executed, the **destructor** `~person()` is called 3 times (in reverse index order from index 2 down to 0).

03 — Memory Leaks, Dangling Pointers, Double Deletion, and RAII



Dangling Pointer and Memory Leak

1. Summary of Dynamic Memory Bugs

1. **Memory Leak**: Overwriting or losing the last pointer to a heap block without calling `delete`.
2. **Dangling Pointer (Use-After-Free)**: Dereferencing a pointer after `delete p;` has already freed the target object.
3. **Double Deletion**: Calling `delete p;` twice on the same heap address.
4. **Mismatched Deallocation**: Mixing `new[]` with `delete`, or `malloc()` with `delete`.

2. Modern C++23 Recommendation

- For a **single dynamic object**, prefer `std::unique_ptr<person> p = std::make_unique<person>();`.
- For a **dynamic array whose size is known at runtime**, prefer `std::vector<int> arr(n);`.

Visual Concept & Memory Diagrams (08 DYNAMIC MEMORY ALLOCATION)

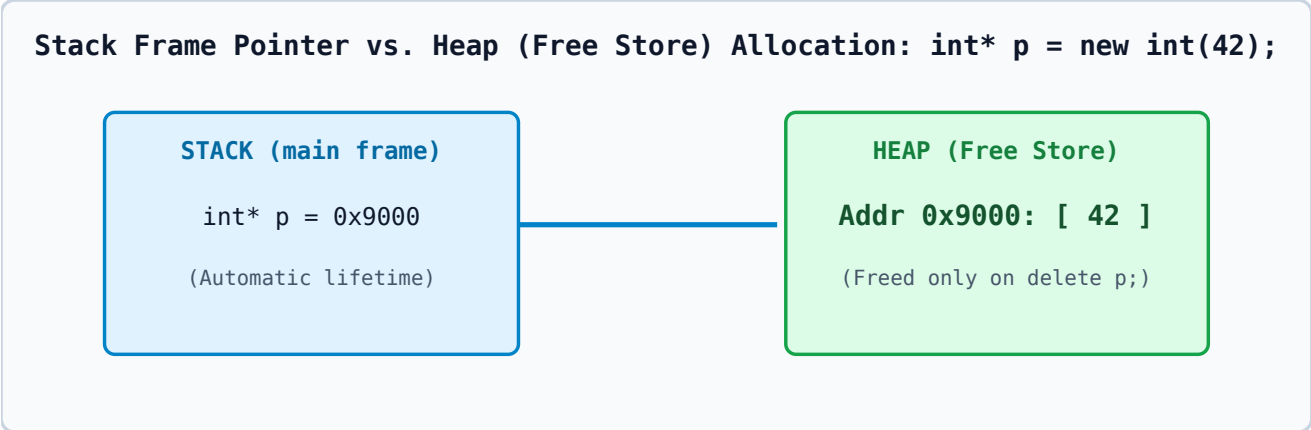


Diagram: diagram 01 stack vs heap memory

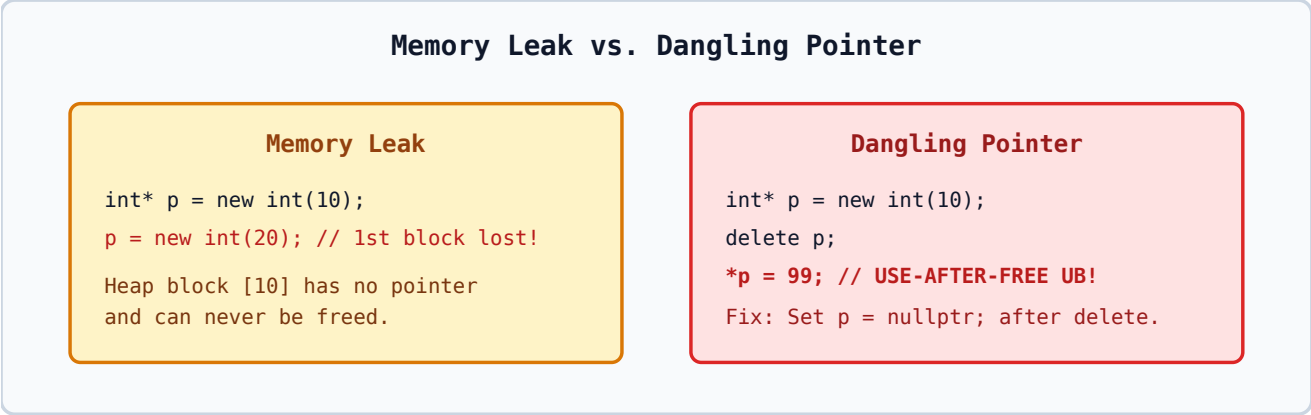


Diagram: diagram 02 dangling and leak

Annotated C++ Programs (08 DYNAMIC MEMORY ALLOCATION)

Program: `01_scalar_and_array_new_delete.cpp`

C++ SOURCE FILE • 01_scalar_and_array_new_delete.cpp

```cpp
// Program Title: Scalar new/delete and Dynamic Array new[]/delete[]
// Topic: Dynamic Memory Allocation
// Purpose: Demonstrate allocating and freeing a single int and a dynamic int array on the heap.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_stack_vs_heap_and_new_delete.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    // 1. Single integer on heap
    int* p = new int(42);
    cout << "Single heap int: " << *p << endl;
    delete p;
    p = nullptr;

    // 2. Dynamic array of integers on heap
    int n = 4;
    int* arr = new int[n]{10, 20, 30, 40};

    cout << "Dynamic heap array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    delete[] arr;
    arr = nullptr;

    return 0;
}

/*
Expected Deterministic Output:
Single heap int: 42
Dynamic heap array: 10 20 30 40
*/
```

Program: `02_dynamic_classroom_person_objects.cpp`

C++ SOURCE FILE • 02_dynamic_classroom_person_objects.cpp

```cpp
// Program Title: Dynamic Allocation of Classroom 'person' Objects
// Topic: Dynamic Memory Allocation — Objects on Free Store
// Purpose: Demonstrate creating and destroying a classroom 'person' object and array of
objects using new and delete[].
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_dynamic_arrays_and_objects.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    person() {
        age = 0;
        cout << "person constructed" << endl;
    }

    ~person() {
        cout << "person destroyed (age=" << age << ")" << endl;
    }

    void input(int a) {
        age = a;
    }

    void display() const {
        cout << "Age: " << age << endl;
    }
};

int main() {
    person* group = new person[2];

    group[0].input(20);
    group[1].input(22);

    group[0].display();
    group[1].display();

    delete[] group;
    group = nullptr;

    return 0;
}

/*
Expected Deterministic Output:
person constructed
person constructed
```

```
Age: 20
Age: 22
person destroyed (age=22)
person destroyed (age=20)
*/
```

Program: `03_mismatched_delete_ub.cpp`

C++ SOURCE FILE • 03_mismatched_delete_ub.cpp

```cpp
// Program Title: Mismatched new[] and Scalar delete Undefined Behavior Explanation
// Topic: Dynamic Memory Allocation — Matching new[] with delete[]
// Purpose: Explain why new int[n] paired with scalar delete (instead of delete[]) is
Undefined Behavior.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_dynamic_arrays_and_objects.md
// Program Classification: Category D — Undefined-behavior demonstration

#include <iostream>
using namespace std;

int main() {
    int* arr = new int[5]{1, 2, 3, 4, 5};
    cout << "First element: " << arr[0] << endl;

    // Correct deallocation uses delete[] arr:
    delete[] arr;

    /*
    NOTE ON UNDEFINED BEHAVIOR (Category D):
    If a program writes:
        int* bad = new int[5];
        delete bad; @@PH_7@@
    ISO C++ [expr.delete] states that if the operand of scalar 'delete' was allocated
    via array 'new[]', the behavior is undefined! Modern compilers (-Wmismatched-new-delete)
    diagnose this bug immediately.
    */
    return 0;
}
```

Program: `04_modern_raii_vs_raw_new.cpp`

C++ SOURCE FILE • 04_modern_raii_vs_raw_new.cpp

```cpp
// Program Title: Classroom Raw new[]/delete[] vs. Modern std::vector
// Topic: Dynamic Memory Allocation — Modern C++ Memory Management
// Purpose: Compare manual dynamic array management with automatic std::vector cleanup.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_memory_bugs_and_raii_ownership.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 3;

    // 1. Classroom raw dynamic array
    int* raw_arr = new int[n]{5, 10, 15};
    cout << "Raw array middle: " << raw_arr[1] << endl;
    delete[] raw_arr;

    // 2. Modern C++ std::vector (automatic heap management, zero leak risk)
    vector<int> vec = {5, 10, 15};
    cout << "Vector middle: " << vec[1] << endl;

    return 0;
}

/*
Expected Deterministic Output:
Raw array middle: 10
Vector middle: 10
*/
```

Conceptual Questions — Module 08: Dynamic Memory Allocation

Q1. Compare `new` / `delete` in C++ with `malloc()` / `free()` in C (mention constructors, destructors, return types, and failure handling). **Q2.** Why must `new int[n]` always be paired with `delete[]` instead of scalar `delete`?

Debugging Questions — Module 08

Q4. Find the memory leak in:

CPP

```cpp
void compute() {
    int* p = new int(5);
    int x = *p * 2;
    p = &x;
    delete p;
}
```

Predict the Output Questions — Module 08

Q3. Predict the output of allocating an array of 3 objects of a class whose constructor prints `C` and destructor prints `D`, followed by `delete[]`.

Programming Questions — Module 08

Q5. Write a C++23 classroom program that dynamically allocates an array of `n` integers, computes their average, and properly deallocates the memory.

Detailed Solutions — Conceptual Questions (Module 08)

- **Q1 Solution**: `new` is an operator that automatically computes size, invokes the class constructor, returns a strongly-typed pointer `T*`, and throws `std::bad_alloc` on failure. `malloc()` is a C library function that only allocates raw uninitialized bytes, does not call constructors, returns `void*`, and returns `NULL` on failure. Similarly, `delete` invokes the destructor before freeing memory, whereas `free()` does not call destructors.
- **Q2 Solution**: `new[]` stores array element count metadata and constructs an array of objects; `delete[]` reads that metadata to invoke the destructor on every element in reverse order before freeing the block. Using scalar `delete` on `new[]` is Undefined Behavior.

Detailed Solutions — Predict the Output & Debugging (Module 08)

- **Q3 Solution**: `new MyClass[3]` calls the constructor 3 times (`C C C`), and `delete[]` calls the destructor 3 times (`D D D`).
- **Q4 Solution**: Two severe bugs: (1) `p = &x;` overwrites the heap address of `new int(5)`, causing a **memory leak**; (2) `delete p;` then attempts to `delete` the address of a stack variable `x`, which is **Undefined Behavior**!

Detailed Solutions — Programming Questions (Module 08)

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    int* arr = new int[n]{10, 20, 30, 40};
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    cout << "Average: " << (sum / n) << endl;
    delete[] arr;
    arr = nullptr;
    return 0;
}
// Expected Output: Average: 25
```